

Министерство науки и высшего образования Российской Федерации

Федеральное автономное образовательное учреждение высшего
образования Национальный исследовательский университет им. Н. И.
Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №6.1

“Морской бой”

Выполнил:

студент группы 3825Б1ФИип2

Богачёв А. В.

Проверил:

Ивлев А. Д.

Нижний Новгород

2026

Содержание

[Введение](#)

[1. Постановка задачи](#)

[2. Руководство пользователя](#)

[3. Руководство программиста](#)

[3.1 Описание структуры программы](#)

[3.2 Описание структур данных](#)

[3.3 Описание алгоритмов](#)

[4. Результаты экспериментов и тестирования](#)

[Заключение](#)

[Литература](#)

[Приложение](#)

Введение

“Морской бой” - игра, в которой требуется расположить собственные корабли разной длины на поле 10x10 клеток. Далее игроки поочередно производят “выстрелы”, называя координаты, соперник сообщает о результате попытки: промах, попадание или затопление корабля.

Задача - разработать игру “морской бой” на языке C++, обеспечив полный игровой процесс человека против компьютера, соблюдая все правила классической игры.

1. Постановка задачи

1. Играют два игрока (человек и компьютер).
2. Каждый игрок имеет в распоряжении флот, состоящий из одного 4-х палубного корабля (авианосца), двух 3-х палубных крейсеров, трех 2-х палубных кораблей и четырех однопалубных катеров.
3. При расстановке корабли не должны соприкасаться, то есть не иметь в координат соседних клеток.
4. Многопалубные корабли могут располагаться только по вертикали или горизонтали.
5. Ходы происходят с соблюдением всех правил классической игры, а первый ход совершает человек. Нельзя делать выстрел в одну и ту же клетку дважды.
6. Выигрывает тот, кто первым потопит все корабли соперника.
7. Для создания игры разработать систему классов, используя известные инструменты языка C++ из курса “Основы программирования” второго семестра первого курса.

2. Руководство пользователя

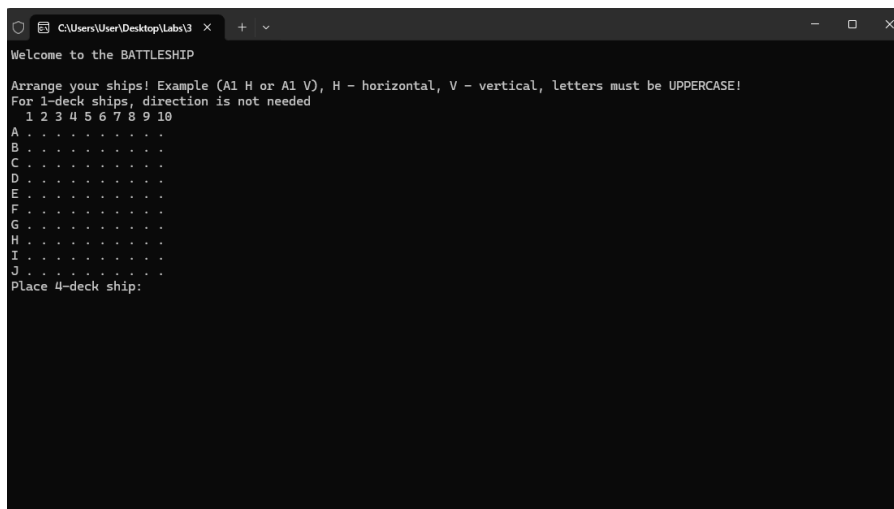


Рис. 1. Начало игры

После запуска игры и приветствия программа предложит игрокам расположить свой флот (компьютер делает расстановку автоматически). Пользователь увидит сообщение, в котором будет сказано, корабль какого размера требуется разместить на поле. Игрок должен ввести координату в формате (A-J)(1-10) (H, V). Заглавная буква и цифра вводятся без пробела, перед направлением корабля H-горизонтально или V-вертикально ожидается ввод пробела). Для однопалубного корабля направление не требуется.

Важное замечание: в буфере обязательно должен присутствовать пробел, иначе может произойти следующая ситуация:

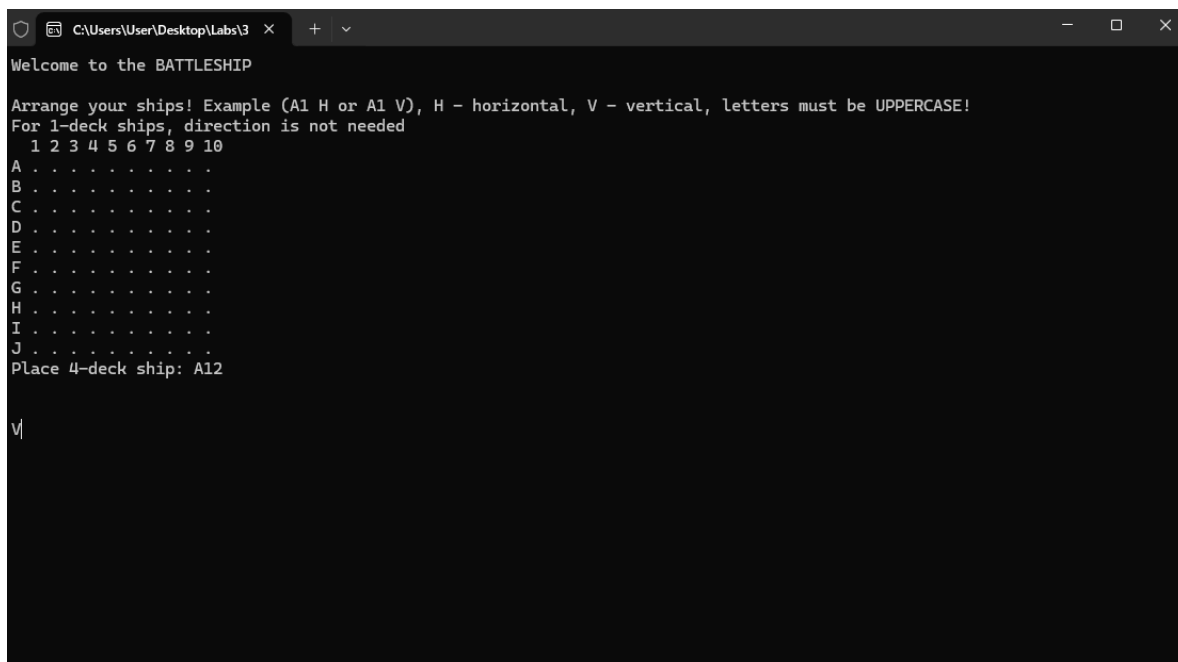


Рис. 2. Отсутствие пробела в потоке ввода

Если пробел (и часть после него) так и не будет введен - игровой процесс не продолжится пока не выполнится соответствующее условие. Поле расстановки состоит из '.', координаты кораблей будут помечены '#'. Как только весь флот будет расставлен, программа даст знать, что игра началась. Человек делает ход первым. Пользователь должен ввести координату в формате (A-J)(1-10) без пробелов. Нельзя совершать выстрел в одну и ту же координату дважды! После выстрела распечатается доска выстрелов игрока, где: '.' - место куда еще не было сделано выстрелов, 'М' - промах, 'Н' - попадание, 'S' - затопленный корабль. Также распечатается результат выстрела. При промахе ход перейдет к компьютеру, результат выстрела противника также отобразится на экране. Главное помнить - корабли не могут соприкасаться палубами, программа не страхует игрока от выстрела в соседнюю клетку от затопленного корабля!

```
C:\Users\User\Desktop\Lab3 x + v
Human, its your turn: A1
Human shoots at A1: Miss.
Your shots against the enemy:
 1 2 3 4 5 6 7 8 9 10
A M . . . . .
B . . . . .
C . . . . .
D . . . . .
E . . . . .
F . . . . .
G . . . . .
H . . . . .
I . . . . .
J . . . . .
Computer shoots at G6: Miss.
Human, its your turn: A2
Human shoots at A2: Hit! Ship sunk!
Your shots against the enemy:
 1 2 3 4 5 6 7 8 9 10
A M S . . . . .
B . . . . .
C . . . . .
D . . . . .
E . . . . .
F . . . . .
G . . . . .
H . . . . .
I . . . . .
J . . . . .
Human, its your turn:
```

Рис. 3. Игровой процесс

После потопления всех кораблей одной из сторон, будет объявлен победитель и игра немедленно завершится.

3. Руководство программиста

3.1 Описание структуры программы

Программа состоит из основных классов которые расположены в одном файле solution 6.1. Её можно условно поделить на несколько основных модулей: Ships, Boards, Players, Game control.

Модуль Ships состоит из основного абстрактного класса Ship от которого наследуются еще четыре класса: Launch, Boat, Cruiser, AircraftCarrier (однопалубный катер, двухпалубный корабль, трехпалубный крейсер, четырехпалубный авианосец) с типом наследования public. Классы хранят координаты корабля, отметки о попаданиях, предоставляют

методы для совершения выстрела, проверки потопления и получения координат. Объекты используются в классе Board.

Модуль Boards содержит в себе два класса, Board и TrackingBoard. Board - основное поле для расстановки кораблей и приёма выстрелов. Содержит “матрицу” 10x10 указателей на объекты класса Ship, отметки о выстрелах противника и список всех кораблей. TrackingBoard - вспомогательное поле для записи результатов выстрелов по противнику. Класс Game хранит объекты данных классов в двух экземплярах, по одному объекту класса на игрока.

Модуль Players содержит базовый класс Player и классы наследники - HumanPlayer и ComputerPlayer (тип наследования public). HumanPlayer реализует взаимодействие с человеком (запрос координат с клавиатуры, ручная расстановка флота). ComputerPlayer реализует поведение компьютера (случайная расстановка и генерация выстрела). Указатели на объекты класса Player используются в классе Game.

Модуль Game control состоит из класса Game, который является главным в программе и полностью обеспечивает игровой процесс. Запускает расстановку, реализует механику хода и перехода хода, проверяет наличие не затопленных кораблей и завершает игру. К модулю также можно отнести функцию main в которой происходит выделение памяти на указатели на игроков и запуск игры.

3.2 Описание структур данных

Глобальные переменные: const int boardSize - обозначает размер поля, массив const char rowLabels[] = {‘A’, ‘B’, ... ‘J’} используется для преобразования введенных пользователем координат, программа ищет символ в массиве и его индекс становится номером строки. Также методы распечатки доски печатают элементы rowLabels, например Board::print().

Структуры: Coord и AttackResult. Coord состоит из двух полей типа int. Имеет конструктор с параметрами по умолчанию, которые используются как признак ошибки, и список инициализации для удобства создания. Применяется для хранения клетки игрового поля с координатами x,y. Используется для передачи координат в методах recieveAttack, record, markSunk, для возврата координаты выстрела в getMove, хранит список неизвестных клеток в ComputerPlayer и используется в методе getUnknown для возврата не стреляной клетки. AttackResult применяется для инкапсуляции результата выстрела по полю противника. Содержит поля bool hit - true если выстрел попал в корабль, иначе false, bool sunk - true если выстрел потопил корабль и указатель на потопленный корабль Ship* sunkShip, nullptr если sunk = false. Возвращается методом recieveAttack, в методе processTurn обрабатывается результат.

Класс Ship и всего наследники содержат поля типа vector для хранения координат, массив отметок о попаданиях bool hits[4] и поле int shipSize - размер корабля. Имеет конструктор, виртуальный деструктор, виртуальный метод возвращения размера (все виртуальные методы будут переопределяться в наследниках), метод setCoords для копирования векторов координат, getCoords для заполнения векторов координатами корабля, методы типа bool: isSunk - узнать потоплен ли корабль, hit - отмечает попадание в клетку, contains - проверяет принадлежность клетки к кораблю.

Класс Board состоит из полей векторов. Матрица(vector<vector>) Ship* grid, матрица отметок о попаданиях bool hit, вектор ship* ships. Содержит конструктор, деструктор, метод проверки на возможность размещения корабля в заданных координатах bool canPlace, метод размещения корабля на поле bool placeShip, AttackResult recieveAttack для обработки выстрела по клетке и возврата результата, bool allSunk для

проверки все ли корабли потоплены, void print для распечатки доски, Ship* getShipAt для возвращения указателя корабля по координатам.

Класс TrackingBoard состоит из поля матрицы char. Имеет конструктор, метод void print для распечатки, void record для записи результата выстрела, void markedSunk для пометки всех клеток потопленного корабля, bool isAttacked для возвращения атакованной клетки.

Классы Player, HumanPlayer, ComputerPlayer. Player - базовый класс содержащий поле string name. Имеет конструктор, виртуальный деструктор, чисто виртуальный метод void placeShips для реализации расстановки кораблей, чисто виртуальный метод Coord getMove для возвращения координаты выстрела, виртуальный метод void resultNotify для уведомления игрока о результате выстрела, string getName для возвращения имени игрока. Классы наследники имеют переопределенные виртуальные методы класса Player, помимо этого имеют собственные методы: HumanPlayer - bool parse для преобразования и проверки на корректность координат введенных пользователем, ComputerPlayer - vector<Coord> getUnknown для возврата координат еще не атакованных клеток.

Класс Game содержит поля указателей на двух игроков, их доски для расстановки и отслеживания атак, bool gameOver - флаг окончания игры. Имеет конструктор и методы управления игрой: void run для начала игры и обеспечения игрового процесса, приватный метод processTurn для обработки хода.

3.3 Описание алгоритмов

Проверка возможности размещения корабля происходит путем “по-клеточной” проверки занятости 8 соседних клеток при помощи цикла for и двух вложенных циклов for в зависимости от направления

расположения корабля (bool horizontal). Параллельно осуществляется проверка выхода за границы поля.

Расстановка человеком: создаются корабли нужных классов с помощью массива размеров. Далее программа запрашивает у пользователя координаты, осуществляется проверка на корректность ввода и возможность расположения. Алгоритм повторяется пока весь флот не будет расставлен.

Расстановка компьютером: генерация случайных координат и случайного направления, осуществляется проверка на возможность расположения. Алгоритм повторяется пока весь флот не будет расставлен. Обработка хода: вызов recieveAttack, запись на TrackingBoard, при потоплении markSunk, Вывод результата в консоль, gameOver = true при победе одной из сторон.

Ход компьютера: сбор всех не атакованных клеток, случайный выбор.

Парсинг: проверка длины ввода, проверка корректности индекса строки, проверка является ли оставшая часть ввода цифрой, преобразование в число.

4. Результаты экспериментов и тестирования

В качестве тестирования были сыграны три игры. Все попытки прошли по ожидаемому сценарию - критических ошибок не выявлено, программа работает корректно.

Намеренно совершались ошибки ввода, которые как и ожидалось на этапе разработки, вызывали сообщение об ошибке и повторный запрос данных.

В проверке правильности расстановки компьютера - нет необходимости, т.к. задействуется тот же алгоритм, что и при расстановке человеком, который во всех трех тестах работал корректно. После победы каждая из трех игр немедленно завершалась.

Заключение

В ходе выполнения лабораторной работы была реализована игра “Морской бой”, были реализованы классы кораблей, полей, полноценный класс игры, а также вспомогательные структуры. Разработан алгоритм расстановки кораблей для компьютера и человека.

Программа полностью обеспечивает игровой процесс и не имеет критических ошибок.

Литература

Сайт Microsoft Learn:

<https://learn.microsoft.com/ru-ru/cpp/cpp/cpp-language-reference?view=msvc-170>

Сайт cppreference: <https://cppreference.com/>

Приложение